

datalab 报告

datalab 报告

姓名：张涵石

学号：2023200068

总分	bitAnd	bitXor	sameSign	logTwo	byteSwap	reverse	logicalShift	leftBitCount	float_i
37.00	1.00	1.00	2.00	4.00	4.00	3.00	3.00	4.00	4.00

test 截图：

测试截图未随 Markdown 文件提供（原路径：report/imgs/test-pass.png）

解题报告

亮点

- 1. byteSwap
- 2. reverse

bitAnd

```
return ~(~x | ~y);
```

这题我想到可以利用德摩根定律，把按位与转换成按位或和取反。先分别对x和y取反，再进行按位或，最后将结果取反，即可得到x&y的结果。从每一位来看，只有x和y对应位都为1时，取反后才都是0，按位或后再取反才会得到1，因此符合按位与的要求。

bitXor

```
return ~(x & y) & ~(~x & ~y);
```

这题需要利用取反和按位与实现异或。我的思路是，异或的结果为1意味着两个对应位不同，也就是既不能同时为1，也不能同时为0。其中 $\sim(x \& y)$ 排除了两个位同时为1的情况， $\sim(\sim x \& \sim y)$ 排除了两个位同时为0的情况，将这两个结果按位与，就只保留了两个位不同的情况，实现了异或。

samesign

```
if (!x) {  
    return !y;  
}  
return !!y && !((x ^ y) >> 31);
```

这题需要特别注意零既不是正数也不是负数，所以不能只比较符号位。我先单独判断x是否为零，如果x为零，就通过!y判断y是否也为零；如果x不为零，则通过!!y排除y为零的情况，再利用 $x \wedge y$ 比较两个数的符号位。如果同号，异或结果的最高位为0，右移31位后得到0，再取逻辑非得到1；如果异号，则得到0。

logtwo

```
int r, s;  
r = ((v >> 16) > 0) << 4;  
v = v >> r;  
s = ((v >> 8) > 0) << 3;  
r = r | s;  
v = v >> s;  
s = ((v >> 4) > 0) << 2;  
r = r | s;  
v = v >> s;  
s = ((v >> 2) > 0) << 1;  
r = r | s;  
v = v >> s;  
return r | (v >> 1);
```

这题我理解为寻找最高位的1所在的位置，也就是一个正整数最多可以右移多少位而不变成零。我按照16、8、4、2、1的顺序尝试右移，如果右移后的结果大于零，就说明可以移动这么多位，于是更新v并记录移动量；否则保持不变，继续尝试更小的步数。由于题目不允许使用加法，而这些移动量分别占据不同

的二进制位，所以可以通过按位或累计移动次数。最后v只可能为1、2或3，因此直接用 $v > 1$ 判断是否还需要记入1位，得到的总次数就是结果。

byteSwap

```
int ns = n << 3;
int ms = m << 3;
int diff = ((x >> ns) ^ (x >> ms)) & 255;
return x ^ (diff << ns) ^ (diff << ms);
```

这题我想到可以借助异或的性质，因为异或满足 $a \wedge (a \wedge b) = b$ 和 $b \wedge (a \wedge b) = a$ 。所以可以首先通过 $n << 3$ 和 $m << 3$ 将字节编号转换为位移量，再将两个目标字节分别右移到最低位，异或后与255按位与，只保留低8位，得到两个字节的差异diff。再将diff分别左移回两个目标位置，与原数异或，就能让两个字节互相变成对方；其他位置与零异或，保持不变。

reverse

```
unsigned result = 0;
int n = 32;
while (n) {
    result = (result << 1) | (v & 1);
    v = v >> 1;
    n = n - 1;
}
return result;
```

这题我的思路是从原数右边依次取出一位，再追加到结果右边。先将result初始化为零，每轮通过 $v \& 1$ 取出原数最低位，将result左移一位腾出位置，再通过按位或放入刚取出的位，然后将v右移一位，丢掉已经处理的位。这样最先取出的原最低位会不断向左移动，最终成为结果的最高位。循环必须执行完整的32次，因为原数高位的零也需要参与反转，不能在v变成零时就提前结束。

logicalShift

```
int mask = ~(((1 << 31) >> n) << 1);
return (x >> n) & mask;
```

这题需要把算术右移转换为逻辑右移。负数进行算术右移时，左侧会补1，而逻辑右移要求补0，所以我先执行 $x >> n$ ，再用掩码清除高位补出的1。构造掩码时，先用 $1 << 31$ 得到只有最高位为1的数，算术右移n位后得到高n+1位为1的数，再左移一位，使高n位为1，最后取反就得到高n位为0、其余位为1的掩

码。将右移结果与掩码按位与，就能保留有效位并清除补出的符号位，n为零时也能保持原数不变。

leftBitCount

```
int y = ~x;
int n, s;
n = !(y >> 16) << 4;
y = y << n;
s = !(y >> 24) << 3;
n = n + s;
y = y << s;
s = !(y >> 28) << 2;
n = n + s;
y = y << s;
s = !(y >> 30) << 1;
n = n + s;
y = y << s;
s = !(y >> 31);
n = n + s;
y = y << s;
return n + !y;
```

这题我先对x取反，将统计左侧连续的1转化为统计左侧连续的0。然后依次检查高16位、高8位、高4位、高2位和最高1位是否全为零，如果为零，就把对应位数计入n，并将y左移这么多位，继续检查后面的部分。例如!(y>>16)可以判断高16位是否全为零，而!(y>>24)可以判断高8位是否全为零。最后通过n+!y处理原数全部为1的特殊情况，此时取反后的y始终为零，前面累计得到31，再加1得到32。

float_i2f

```
unsigned sign = 0;
unsigned v = x;
unsigned exponent = 158;
unsigned fraction, tail;
if (!x) {
    return 0;
}
if (x < 0) {
    sign = 0x80000000;
    v = ~v + 1;
}
while (!(v >> 31)) {
    v = v << 1;
    exponent = exponent - 1;
}
fraction = (v >> 8) & 0x7ffffff;
tail = v & 255;
if (tail > 128) {
    fraction = fraction + 1;
} else if (tail == 128) {
    fraction = fraction + (fraction & 1);
}
return sign + (exponent << 23) + fraction;
```

这题需要按照单精度浮点数的格式，分别构造符号位、指数和尾数。我先单独处理零，再提取符号，并在无符号变量中求出整数的绝对值，避免最小负数取绝对值时超出有符号整数范围。接着不断左移 v ，直到最高位为1，指数从158开始随每次左移减1，其中158等于指数偏置127加31。之后去掉隐含的最高位1，保留后面的23位作为尾数，并根据舍弃的低8位进行舍入：大于128时进位，小于128时不进位，等于128时根据保留部分的最低位决定是否进位，实现就近舍入且中间值取偶数。最后用加法组合各部分，让尾数舍入产生的进位可以自然进入指数。

floatScale2

```
unsigned sign = uf & 0x80000000;
unsigned exponent = (uf >> 23) & 255;
unsigned fraction = uf & 0x7fffff;
if (exponent == 255) {
    return uf;
}
if (!exponent) {
    return sign | (fraction << 1);
}
exponent = exponent + 1;
if (exponent == 255) {
    return sign | 0x7f800000;
}
return sign | (exponent << 23) | fraction;
```

这题我先将浮点数拆成符号、指数和尾数，再根据指数分类处理。如果指数为255，说明是无穷大或NaN，直接返回原值；如果指数为零，说明是零或非规格化数，只需要将尾数左移一位并保留符号，其中非规格化数也可能通过这次左移自然转为规格化数。对于普通规格化数，乘以2相当于指数加1，尾数不变。如果增加后的指数达到255，则发生溢出，需要将尾数清零，返回对应符号的无穷大。

float64_f2i

```
int exponent = (uf2 >> 20) & 0x7fff;
unsigned sign = uf2 >> 31;
unsigned significand = (uf2 & 0xfffff) | 0x100000;
unsigned magnitude;
exponent = exponent - 1023;
if (exponent < 0) {
    return 0;
}
if (exponent >= 31) {
    return 0x80000000;
}
if (exponent <= 20) {
    magnitude = significand >> (20 - exponent);
} else {
    magnitude = (significand << (exponent - 20)) |
                (uf1 >> (52 - exponent));
}
if (sign) {
    return ~magnitude + 1;
}
return magnitude;
```

这题需要把分成高低两部分的双精度浮点数转换为整数。我先从uf2中提取符号和指数，减去偏置1023得到实际指数，再将高20位尾数与隐含的最高位1组合起来。如果实际指数小于零，说明绝对值小于1，向零取整后返回0；如果指数大于等于31，则返回0x80000000，其中合法的最小负整数也恰好具有这一位模式。对于其余情况，如果指数不超过20，直接右移高位有效数字就能得到整数部分；如果超过20，则需要将高位有效数字左移，并从uf1中补入剩余的整数位。最后根据符号决定是否取补码，通过先截取大小的整数部分再恢复符号，实现向零取整。

floatPower2

```
if (x < -149) {  
    return 0;  
}  
if (x < -126) {  
    return 1 << (x + 149);  
}  
if (x > 127) {  
    return 0x7f800000;  
}  
return (x + 127) << 23;
```

这题的结果是2的整数次幂，二进制有效数字只有一个1，所以我按照指数范围分类构造浮点数。如果x小于-149，结果比最小非规格化数还小，返回0；如果x在-149到-127之间，就用非规格化数表示，因为最低尾数位代表2的-149次方，所以将1左移x+149位即可。如果x在-126到127之间，则使用规格化数表示，尾数为零，只需要将加上偏置127后的指数左移23位。如果x大于127，结果超出单精度浮点数范围，返回正无穷。

bitSwap

```
int ns = n << 3;  
int ms = m << 3;  
int diff = ((x >> ns) ^ (x >> ms)) & 255;  
return x ^ (diff << ns) ^ (diff << ms);
```

反馈/收获/感悟/总结

这次实验对我来说还是有一定难度的，整体花了不少时间。做每道题时，我常能想到一个大致的方向，但难以真正写成代码。这种情况下还是得请教AI，让它帮我补充一下，才能最后把代码搞出来。

参考的重要资料